Name : Chaudhari nilesh **Roll No: COA06**
Div : A **Class : BE**

ASSIGNMENT NO:1

Program to display the Fibonacci sequence up to n-th term

```
nterms = int(input("Enter number of terms "))# first two terms

n1, n2 = 0, 1

count = 0

# check if the number of terms is validif nterms <= 0:

print("Please enter a positive integer")# if there is only
one term, return n1

elif nterms == 1:

print("Fibonacci sequence upto", nterms,":")print(n1)

# generate fibonacci sequenceelse:

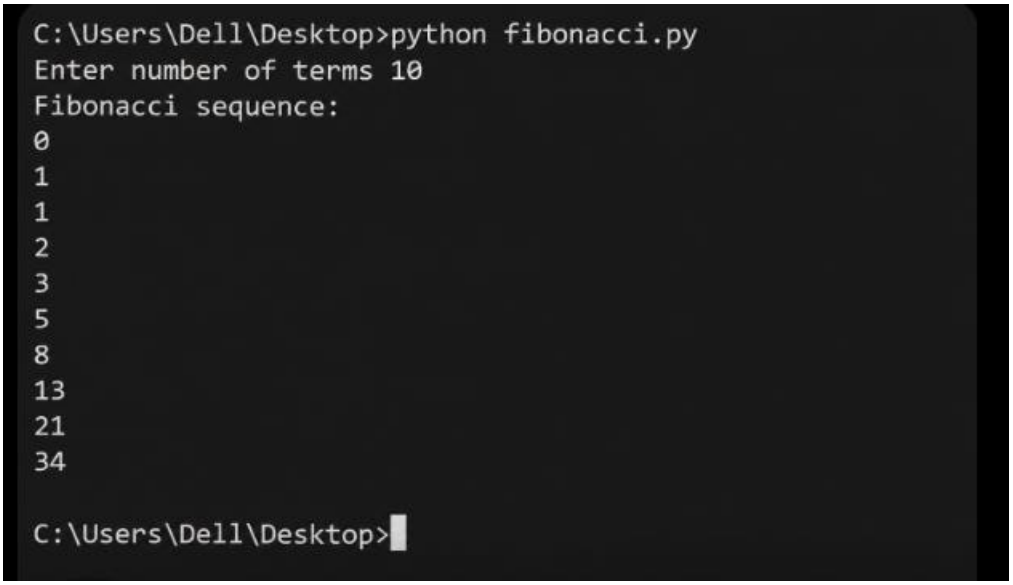
print("Fibonacci  sequence:") while  count  <
nterms:print(n1)

nth = n1 + n2

n2 = nth

n1=n2

count+=1
```


C:\Users\Dell\Desktop>python fibonacci.py
Enter number of terms 10
Fibonacci sequence:
0
1
1
2
3
5
8
13
21
34
C:\Users\Dell\Desktop>

Name : Chaudhari nilesh **Roll No: COA06**
Div : A **Class : BE**

ASSIGNMENT NO:2

```
def printNodes(node, val=''):
        newVal = val +
        str(node.huff)
        if(node.left):

                printNodes(node.left
        , newVal)if(node.right):

                printNodes(node.right,
        newVal) if(not node.left and
        not node.right):

                print(f"{node.symbol} -> {newVal}")

# characters for huffman tree
                chars = ['a', 'b', 'c', 'd', 'e', 'f', 'g']

# frequency of characters

freq = [ 4, 7, 12, 14, 17, 43, 54]

# list containing unused nodes

nodes = []

# converting characters and frequencies into huffman tree nodes

for x in range(len(chars)):

nodes.append(node(freq[x], chars[x]))

while len(nodes) > 1:

# sort all the nodes in ascending order based on their frequency

nodes = sorted(nodes, key=lambda x: x.freq)

# pick 2 smallest nodes

left = nodes[0]
```

```
right = nodes[1]

# assign directional value to these nodes

left.huff = 0

right.huff = 1

# combine the 2 smallest nodes to create new node as their parent

                newNode = node(left.freq+right.freq, left.symbol+right.symbol, left,
        right)

        # remove the 2 nodes and add their parent as new node among others

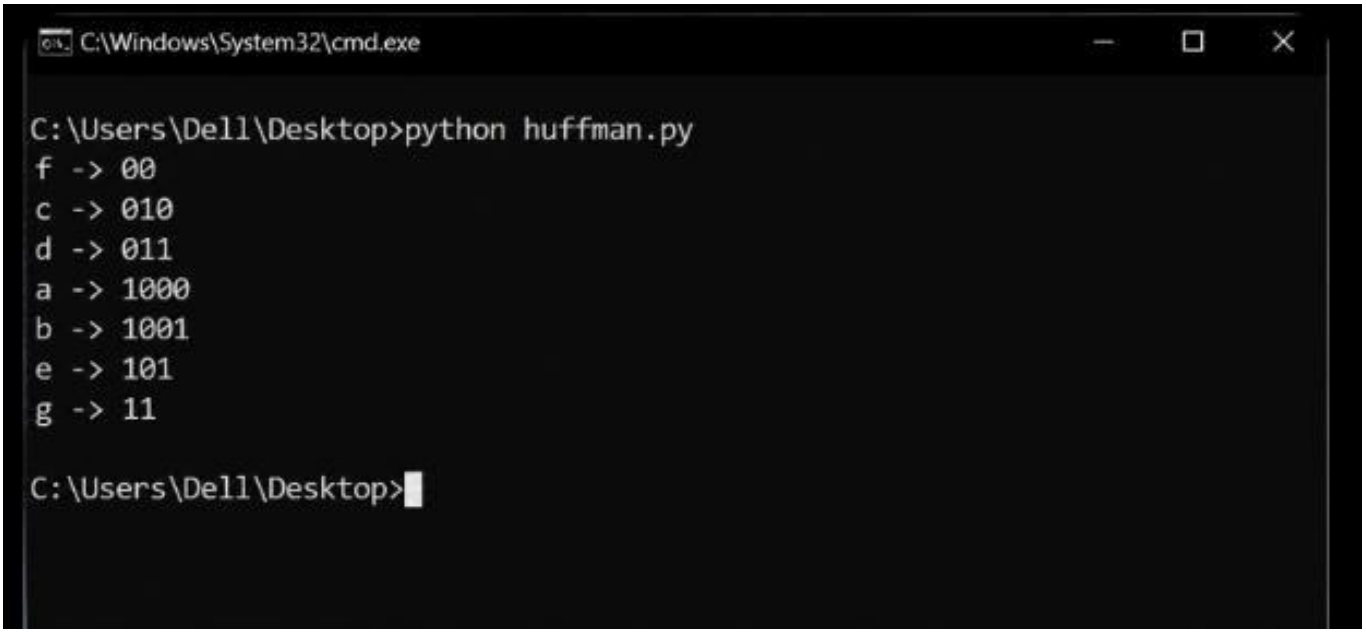
        nodes.remove(left)

        nodes.remove(right)

        nodes.append(newNode)

                # Huffman Tree is ready!

                printNodes(nodes[0])
```



Name : Chaudhari nilesh **Roll No: COA06**

Div : A **Class : BE**

ASSIGNMENT NO:3

```
• def fractional_knapsack(value, weight, capacity):

# index = FRACTIONAL KNAPSACK PROBLEM USING GREEDY ALGORITHM [0, 1, 2, ..., n - 1

# contains ratios of values to weight

ratio = [v/w for v, w in zip(value, weight)]

# index is sorted according to value-to-weight ratio in decreasing order

index.sort(key=lambda i: ratio[i], reverse=True)

max_value = 0

fractions = [0]*len(value)

for i in index:

if weight[i] <= capacity:

fractions[i] = 1

max_value += value[i]

capacity -= weight[i]

else:

fractions[i] = capacity/weight[i]

max_value += value[i]*capacity/weight[i]

break

return max_value, fractions

n = int(input('Enter number of items: '))

value = input('Enter the values of the {} item(s) in order: '.format(n)).split()

value = [int(v) for v in value]

weight = input('Enter the positive weights of the {} item(s) in order:
'.format(n)).split()

weight = [int(w) for w in weight]
```

```
capacity = int(input('Enter maximum weight: '))

max_value, fractions = fractional_knapsack(value, weight, capacity)

print('The maximum value of items that can be carried:', max_value)

print('The fractions in which the items should be taken:', fractions)
```

```
C:\Users\Dell\Desktop>python fractional_knapsack.py
Enter number of items 4
Enter the values of the 4 item(s) in order: 60 100 120 130
Enter the positive weights of the 4 item(s) in order: 10 20 30 40
Enter maximum weight: 50

The maximum value of items that can be carried: 240.0
The fractions in which the items should be taken: [1, 1, 1.0, 0.5]

C:\Users\Dell\Desktop>
```

Name : Chaudhari nilesh **Roll No: COA06**

Div : A **Class : BE**

ASSIGNMENT NO:4

```
def knapsack(weights, values, capacity, n):
    dp = [[0 for _ in range(capacity + 1)]
          for _ in range(n + 1)]
    for i in range(1, n + 1):
        for w in range(capacity + 1):
            if weights[i - 1] <= w:
                dp[i][w] = max(
                    values[i - 1] + dp[i - 1][w - weights[i - 1]],
                    dp[i - 1][w]
                )
            else:
                dp[i][w] = dp[i - 1][w]

    selected = []
    w = capacity
    for i in range(n, 0, -1):
        if dp[i][w] != dp[i - 1][w]:
            selected.append(i)
            w -= weights[i - 1]

    selected.reverse()
    return dp[n][capacity], selected

n = int(input("Enter number of items: "))
values = list(map(int, input("Enter values of items: ").split()))
weights = list(map(int, input("Enter weights of items: ").split()))
capacity = int(input("Enter knapsack capacity: "))

max_value, selected = knapsack(weights, values, capacity, n)

print("Maximum value:", max_value)
print("Selected item numbers:", selected)
```

```
C:\Users\Dell\Desktop>python knapsack.py

Enter number of items: 4
Enter values of items: 40 50 100 95
Enter weights of items: 2 3 5 5
Enter knapsack capacity: 10

Maximum value: 195
Selected item numbers: [2, 3, 4]

C:\Users\Dell\Desktop>
```

Name : Chaudhari nilesh **Roll No: COA06**

Div : A **Class : BE**

ASSIGNMENT NO:5

```
#include <bits/stdc++.h>
using namespace std;
#define N 4
bool isSafe(int board[N][N], int row, int col) {
   int i, j;
   for (i = 0; i < col; i++)
      if (board[row][i])
         return false;
   for (i = row, j = col; i >= 0 && j >= 0; i--, j--)
      if (board[i][j])
         return false;
   for (i = row, j = col; i < N && j >= 0; i++, j--)
      if (board[i][j])
         return false;
   return true;
}
bool solveNQUtil(int board[N][N], int col) {
   if (col >= N)
      return true;
   for (int i = 0; i < N; i++) {
      if (isSafe(board, i, col)) {
         board[i][col] = 1;
         if (solveNQUtil(board, col + 1))
            return true;
         board[i][col] = 0;
      }
   }
   return false;
void printSolution(int board[N][N]) {
   cout << "Solution for 4-Queens Problem:" << endl;
   for (int i = 0; i < N; i++) {
      for (int j = 0; j < N; j++)
```

```
            cout << board[i][j] << " ";
        cout << endl;
    }
}
bool solveNQ() {
    int board[N][N] = {0};
    if (!solveNQUtil(board, 0)) {
        cout << "Solution does not exist";
        return false;
    }
    printSolution(board);
    return true;
}
int main() {
    solveNQ();
    return 0;
}
```

```
C:\Users\Dell\Desktop>g++ NQueens.cpp -o NQueens

C:\Users\Dell\Desktop>NQueens.exe

Solution for 4-Queens Problem:
0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0

C:\Users\Dell\Desktop>
```